

Deployment Document

Smart Building Monitoring — Facilities Operations Dashboard

CET333 Product Development · Task 7 (Deployment)

Next.js 16 · React 19 · TypeScript · Firebase Authentication and Cloud Firestore

Contents

- 1. How the system will be deployed
- 2. Hardware requirements
- 3. Software requirements
- 4. Installation procedure
- 5. Configuration
- 6. User documentation
- 7. Maintenance considerations

1. How the system will be deployed

The system deploys as two halves that are hosted separately and never share a machine.

The first half is the application itself: a Next.js front end compiled to static HTML, CSS and JavaScript. Sixteen of its seventeen routes are prerendered at build time and served as files from a content delivery network. Only the catch-all 404 route is rendered on demand. There is no application server to provision, patch or scale.

The second half is the data: Firebase Authentication and Cloud Firestore, both managed by Google. Every page opens a live onSnapshot subscription, so a change made by one person appears on another person's screen without a refresh and without any polling code. Authorisation is enforced by Firestore security rules, which run on Google's servers rather than in the browser.

The practical consequence is that deployment is a file upload plus a rules deployment. There is no database to install, no connection string, no server operating system, and nothing listening on a port that the university would have to firewall.

1.1 Deployment topology

Layer	What is deployed	Where it runs	Who operates it
Client	Compiled HTML, CSS, JavaScript (3.2 MB static, 2.6 MB JavaScript)	The end user's web browser	The user
Hosting	The same compiled output, served over HTTPS	Vercel or Firebase Hosting CDN	The hosting provider
Authentication	Email and password identity, session tokens	Firebase Authentication (Google)	Google
Database	Eleven Firestore collections	Cloud Firestore, europe-west or nearest region	Google
Authorisation	firestore.rules	Cloud Firestore rules engine	Deployed by the administrator

1.2 Deployment environments

Environment	Purpose	Data	How it is started
Local development	Day-to-day coding	The developer's own Firebase project	pnpm dev
Local emulator	Testing rules and destructive actions safely	Throw-away, in memory	firebase emulators:start
Production	The live system	The live Firebase project	pnpm build, then deploy

2. Hardware requirements

Because the application is a static front end talking to a managed database, there is no server hardware to specify. The requirements fall into two groups: the machine used to build and deploy the system, and the devices used to run it.

2.1 Build and deployment machine

Needed once per release, by whoever publishes the build. Not needed to use the system.

Component	Minimum	Recommended	Why
Processor	Dual-core x86-64 or Apple Silicon, 2.0 GHz	Quad-core, 2.5 GHz or faster	A production build compiles seventeen routes; on a dual-core machine this takes noticeably longer
Memory	4 GB RAM	8 GB RAM	Node holds the whole module graph in memory during the build. 4 GB completes but leaves little headroom
Storage	3 GB free	5 GB free	node_modules is roughly 700 MB and the build cache grows to several gigabytes over repeated builds
Display	1280 x 720	1920 x 1080	The desktop layout is designed at 1440 px and above
Network	Any broadband connection	10 Mbps or better	Dependency installation downloads roughly 700 MB the first time

2.2 End-user devices

The interface is built for three widths and was tested at each. No installation is required on any of them.

Device class	Screen width	Layout served	Minimum specification
Desktop or laptop	1024 px and above	Full 196 px sidebar, multi-column pages	Any machine from roughly 2016 onward running a current browser; 4 GB RAM
Tablet	768 to 1023 px	60 px icon rail, condensed tables	iPad from 2018 onward, or an equivalent Android

			tablet
Phone	Below 768 px	Bottom tab bar, single-column, filters in a bottom sheet	Any phone running a browser released in the last three years

A touch screen is supported but not required; every action is reachable with a keyboard and pointer. No printer is required, although the Reports page produces a print-ready PDF through the browser's own print dialogue.

2.3 Server and network hardware

Item	Requirement
Application server	None. The front end is static and served from a CDN
Database server	None. Cloud Firestore is a managed service
Storage volume	None. Photographs are stored as compressed data inside Firestore documents, capped at 700 KB each
Fixed IP or DNS	Only a domain name, if a custom one is wanted. The hosting provider supplies a working address without it
Firewall change	None inbound. Client devices need outbound HTTPS on port 443 to the domain, firebaseapp.com and googleapis.com

3. Software requirements

3.1 Build environment

Software	Version used	Minimum	Purpose
Node.js	22.23.1	20.0 or newer	Runs the compiler and the tooling
pnpm	11.18.0	9.0 or newer	Installs dependencies. npm also works
Git	Any current version	2.30	Retrieving the source
Firebase CLI	13.35.1	13.0	Deploying the security rules and running the emulator
Operating system	Fedora Linux	Linux, macOS 12+, or Windows 10+ with WSL2	Any of the three is supported

3.2 Application dependencies

Installed automatically by the install command. Versions are pinned in pnpm-lock.yaml so every machine resolves the same tree.

Package	Version	What it provides
next	16.3.4	Framework, App Router and the build pipeline
react / react-dom	19.2.8	The user interface library
firebase	12.19.0	Authentication and Firestore client SDK
@base-ui/react	1.8.0	Accessible dialog, select and sheet primitives used by shadcn/ui
recharts	3.10.1	Sensor reading charts and the dashboard load profile
motion	13.4.1	Card movement between board columns, and exit animations
lucide-react	1.43.0	Icon set
sonner	2.0.8	Toast notifications
next-themes	0.4.6	Light, dark and system theme switching
tailwindcss	4.x	Styling, via the design token sheet in globals.css
typescript	5.x	Static typing across the whole

		source tree
biome	2.x	Linting and formatting
vitest	5.0.1	Unit tests, 272 of them over the pure rules in lib/

3.3 End-user software

Requirement	Detail
Web browser	Chrome, Edge, Firefox or Safari, any version from the last two years. The build targets browsers supporting ES2022 and CSS nesting
JavaScript	Must be enabled. The application will not function without it
Cookies	Not used. The session is held in sessionStorage and ends when the tab closes
Screen reader	Optional. Interactive controls carry ARIA labelling and every status is conveyed by icon and word as well as colour
Additional software	None. There is nothing to install on a user device

3.4 Cloud services

Service	Plan	What it is used for	Free-tier limit
Firebase Authentication	Spark (free)	Email and password sign-in for all staff accounts	Unlimited for email and password
Cloud Firestore	Spark (free)	Eleven collections of live application data	1 GiB stored, 50,000 reads and 20,000 writes per day
Hosting (Vercel or Firebase)	Hobby / Spark	Serving the compiled front end	100 GB bandwidth per month

Note on capacity. Every page holds live subscriptions, so a read is charged per document delivered. With three buildings and around thirty concurrent users the estimated daily read count stays comfortably inside the free tier. A larger estate would need the Blaze pay-as-you-go plan.

4. Installation procedure

Roughly fifteen minutes on a machine that already has Node and Git. Every command is run from a terminal in the project folder unless stated otherwise.

Step 1 — Obtain the source

```
git clone <repository-url>
cd smart-campus-management
```

Step 2 — Install the runtime

Confirm Node is version 20 or newer, then install pnpm if it is not present.

```
node -v      # must print v20.x or higher
npm install -g pnpm
pnpm -v
```

Step 3 — Install dependencies

Downloads roughly 700 MB the first time. Later runs are served from the pnpm store and take seconds.

```
pnpm install
```

Step 4 — Create the Firebase project

1. Open console.firebase.google.com and choose Add project. Any name is acceptable; Google Analytics can be turned off.
2. Open Build, then Authentication, then Get started. Enable the Email/Password provider and save.
3. Open Build, then Firestore Database, then Create database. Choose the region closest to the users. Start in production mode; the rules are deployed in Step 7.
4. Open Project settings, then General, then Your apps. Click the web icon, register an app with any nickname, and leave the firebaseConfig block on screen for the next step.

Step 5 — Supply the project credentials

Copy the example file and fill in the six values from the firebaseConfig block.

```
cp .env.example .env.local
```

Key in .env.local	Value from firebaseConfig
NEXT_PUBLIC_FIREBASE_API_KEY	apiKey
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	authDomain
NEXT_PUBLIC_FIREBASE_PROJECT_ID	projectId
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	storageBucket
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	messagingSenderId
NEXT_PUBLIC_FIREBASE_APP_ID	appId

These six values are not secret. They identify which project the browser should talk to and are present in the compiled JavaScript by design. What protects the data is the security rules deployed in Step 7, not the secrecy of these keys.

Step 6 — Create the accounts and the starting estate

This writes the three role accounts and the estate they manage. It must run before the security rules are deployed, because the first user profile cannot be written through those rules.

```
INIT_PASSWORD='Password!2026' pnpm init:project
```

It creates three buildings, nineteen rooms, sixteen equipment types, twenty-eight equipment units, five sensor types and twenty-nine sensors. It deliberately creates no maintenance requests, log entries or reports, so the record starts clean and fills as the system is used.

Account	Role	Scope
daw.htun@university.edu	CEO / Super Admin	The whole estate
elysha@university.edu	Admin Manager	The whole estate, except buildings and rooms
hnin.nwe@university.edu	Office Staff	Building 216 only

Step 7 — Deploy the security rules

Until this runs, the database is open to anyone holding the configuration values above. This step is what makes the role restrictions real rather than cosmetic.

```
firebase login
firebase use --add      # select the project created in Step 4
pnpm rules:deploy
```

Step 8 — Verify locally

```
pnpm dev
```

Open <http://localhost:3000> and sign in as the CEO account. Check that the dashboard draws, the sidebar shows eight items, and the estate table lists the three buildings.

Step 9 — Build for production

Verify the build by its exit code. The message about successful compilation is printed before the prerender step, which can still fail after it.

```
pnpm build
echo $?      # must print 0
```

Step 10 — Publish

Either hosting route serves the same compiled output.

Option	Commands	Notes
Vercel	<code>npm i -g vercel, then vercel --prod</code>	Detects Next.js automatically. The

		six environment variables must be added in the Vercel dashboard
Firebase Hosting	firebase init hosting, then firebase deploy --only hosting	Keeps hosting and database in one console

Step 11 — Post-deployment check

1. Sign in as each of the three accounts in turn.
2. Confirm Office Staff sees five sidebar items and no building selector.
3. Confirm Log Book, Reports and Administration each show a refusal for Office Staff.
4. Raise one test request and confirm it appears in Firestore.
5. Delete the test request and run the reset command if a clean start is wanted.

5. Configuration

5.1 Environment variables

All configuration is supplied through environment variables. There is no configuration file to edit inside the source tree.

Variable	Required	Default	Effect
NEXT_PUBLIC_FIREBASE_API_KEY	Yes	none	Identifies the Firebase project
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	Yes	none	Domain used for the sign-in flow
NEXT_PUBLIC_FIREBASE_PROJECT_ID	Yes	none	Which Firestore database to read
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	Yes	none	Required by the SDK even though no bucket is used
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	Yes	none	Required by the SDK
NEXT_PUBLIC_FIREBASE_APP_ID	Yes	none	Identifies this web app within the project
NEXT_PUBLIC_FIREBASE_EMULATORS	No	0	Set to 1 to talk to the local emulator suite instead of the live project
INIT_PASSWORD	No	SmartPassword!	Shared password given to the three accounts created by the setup script

The application fails loudly at load if the API key is missing, rather than surfacing the problem much later as a confusing sign-in error:

```
if (!useEmulators && !firebaseConfig.apiKey) {  
  throw new Error(  
    "Firebase is not configured. Copy .env.example to .env.local and fill in the " +  
    "NEXT_PUBLIC_FIREBASE_* values, or set NEXT_PUBLIC_FIREBASE_EMULATORS=1 to " +  
    "run against the local emulator suite.",  
  );  
}  
  
// src/lib/firebase.ts
```

5.2 Database structure

Eleven collections, each behind a live subscription. Firestore is schemaless, so the shape below is enforced by TypeScript in `src/lib/types.ts` and by the security rules, not by the database itself.

Collection	Document id	Holds	Written by
users	Firebase uid	Name, email, role, assigned building, status	Administration
buildings	Human id, e.g. b216	Name, site code, address, floors, description	Administration (CEO only)
rooms	Human id, e.g. r-216-302	Name, floor, room type, parent building	Administration (CEO only)
equipmentTypes	Slug, e.g. air-conditioner	Label and archive state	Equipment, type registry
equipmentUnits	Asset tag, e.g. EQ-216-01	Type, room, condition, install date, service interval	Equipment
equipmentHistory	Auto id	One row per thing that happened to a unit	Equipment, batched with the change
sensorTypes	Slug, e.g. fire-alarm	Statuses, actions, measurement bands, room-type exceptions	Sensors, type registry
sensors	Device id, e.g. FD-216-14	Type, room, current status, last reported time	Sensors
requests	Request number, e.g. REQ-4192	Issue, priority, status, submitter, decline reason	Requests
logBook	Auto id	Append-only audit trail of every action	Every page
reports	Report id	A whole generated report, snapshotted at generation	Reports

Two conventions are worth noting. The human identifier is the document id wherever one exists, because every cross-reference already holds that string. And `logBook` and `equipmentHistory` are append-only: an entry is never edited once written, because an audit trail that can be rewritten is not one.

5.3 Indexes

firebase.indexes.json is deliberately empty. Every query is single-collection and ordered on one field, which Firestore serves from its automatic single-field indexes. Adding a where clause alongside an orderBy on a different field would require a composite index and a redeployment.

```
// src/lib/firestore-store.ts
export const COLLECTIONS = {
  users: "users",
  logBook: "logBook",
  buildings: "buildings",
  rooms: "rooms",
  sensorTypes: "sensorTypes",
  sensors: "sensors",
  equipmentUnits: "equipmentUnits",
  equipmentTypes: "equipmentTypes",
  equipmentHistory: "equipmentHistory",
  requests: "requests",
  reports: "reports",
} as const;
```

5.4 Roles and authorisation

Three roles, ranked. Permissions are decided by rank rather than by comparing role names, so a new role slots in without rewriting every check.

Role	Rank	May do
CEO / Super Admin	1	Everything, including buildings, rooms and all accounts
Admin Manager	2	Everything except buildings and rooms; may manage Office Staff accounts only
Office Staff	3	Their own building. Raise, withdraw and verify requests; act on equipment except decommissioning. Sensors are read-only. No Log Book, Reports or Administration

```
// src/lib/permissions.ts
export function canAct(role: UserRole): boolean {
  return role !== "office-staff";
}

export function canManageEstate(role: UserRole): boolean {
  return roleRank(role) <= 1; // CEO only
}
```

Two layers say the same thing. `src/lib/permissions.ts` decides what to draw on screen; `firestore.rules` decides what the database will actually allow. When one changes, the other must change in the same commit, or the padlocks become decoration.

5.5 Theme and appearance

Light, dark and system themes are switched on the Settings page and remembered per signed-in user. Colours are CSS variables declared in `src/app/globals.css`; no colour is written as a literal anywhere in the source.

6. User documentation

A step-by-step guide to each page, with the screen as the user sees it and the source file that produces it. Every page lives in one file, with its sub-components beside it.

6.1 Signing in

1. Open the application address in a browser.
2. Enter the university email address and password.
3. Select Sign in.

The session lasts until the browser tab is closed. It is held in sessionStorage rather than a cookie, so closing the tab signs you out and two tabs do not share a session. If the password is wrong the message is deliberately vague, so the form cannot be used to discover which addresses exist.

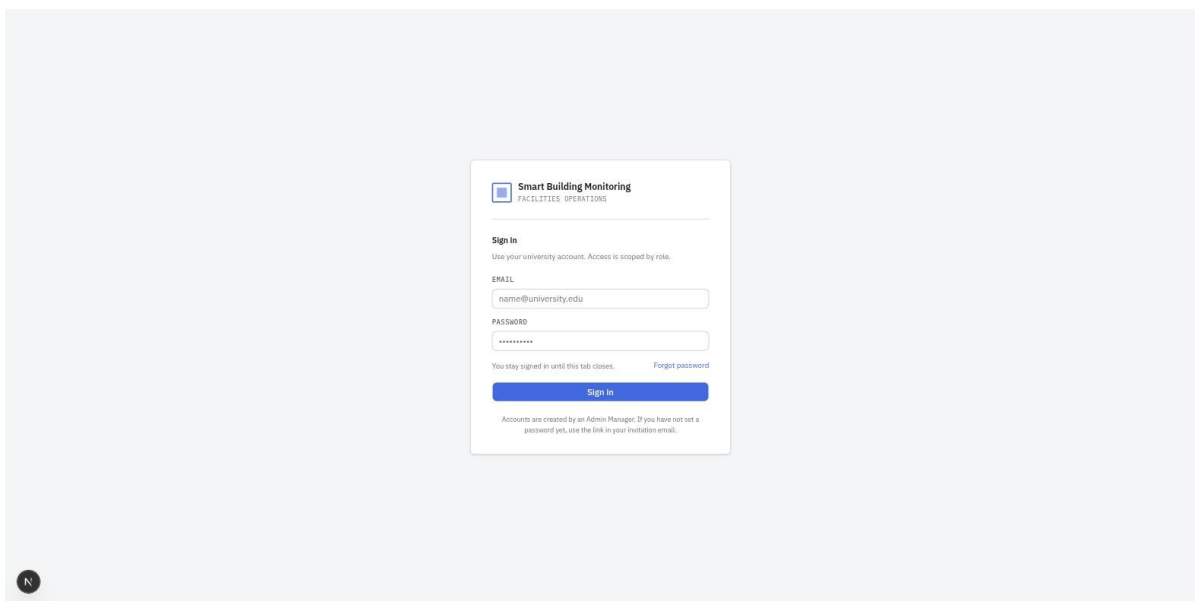


Figure 1 — the sign-in screen

```
// src/app/login/page.tsx — the sign-in form
// src/lib/auth.tsx — useAuth(), the only module importing firebase/auth
```

6.2 Dashboard

The opening screen. It answers what needs attention right now.

1. Choose a building from the selector at the top left to narrow every figure on the page. Office Staff do not see this control; they are scoped to their own building.
2. Read the equipment tiles for running, under maintenance and faulty counts.
3. Use Requests needing a decision to advance a request one step without leaving the page.
4. Watch Live alerts and the Log Book rail on the right, both of which update without a refresh.

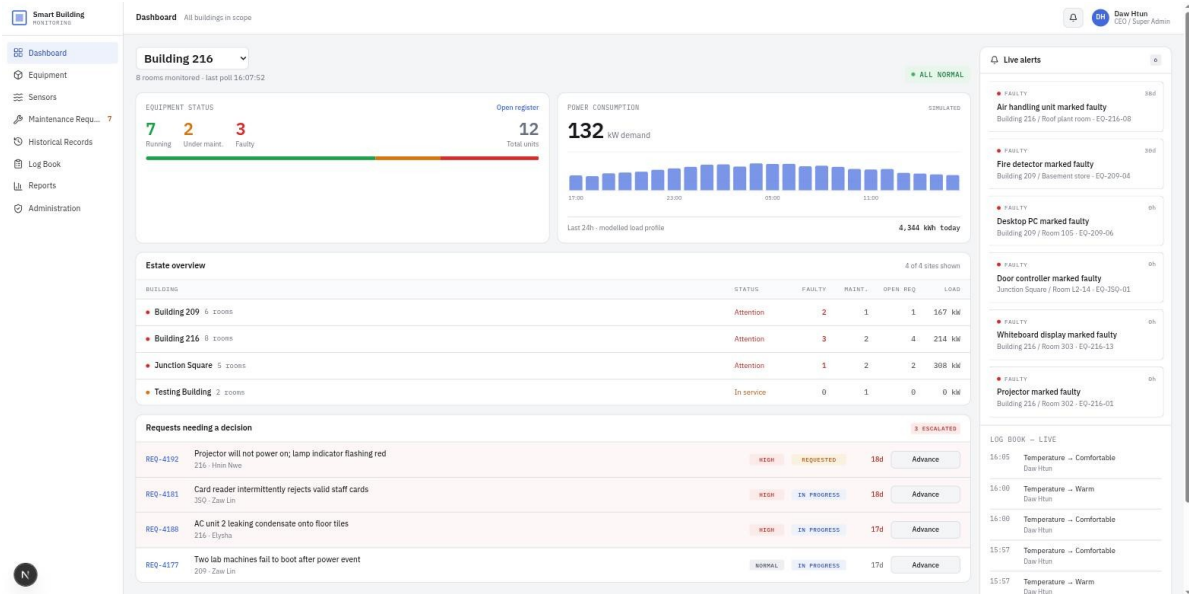


Figure 2 — the dashboard, whole estate

```
// src/app/(app)/dashboard/page.tsx
```

6.3 Equipment register

Every asset the estate owns, in a table or on a condition board.

1. Switch between Register and Condition board with the control beside the search box.
2. Narrow the list by search text, building, or equipment type.
3. Turn on Show decommissioned to include retired units, which are hidden by default.
4. Select any row to open its detail drawer.
5. In the drawer, use one of the six actions: Mark faulty, Return to service, Under maintenance, Record a service, Move unit, or Decommission.
6. Decommissioning and deleting both demand a written reason before they will proceed.

TAG	TYPE	LOCATION	CONDITION	INSTALLED	NEXT SERVICE	REQUESTS	DETAIL
EQ-209-01	Projector	Workshop 1.04 - Building 209	HEALTHY	11 May 2023	24 Nov 2026	0	Open -
EQ-209-02	Air conditioner	Atrium - Building 209	HEALTHY	20 Sept 2021	03 Oct 2026	0	Open -
EQ-209-03	Door controller	Loading bay - Building 209	UNDER MAINTENANCE	08 Mar 2022	16 Mar 2027	0	Open -
EQ-209-04	Fire detector	Basement store - Building 209	FAULTY	22 Jul 2020	27 Aug 2026	0	Open -
EQ-209-06	Desktop PC	Room 105 - Building 209	FAULTY	11 Oct 2023	11 Oct 2026	1	Open -
EQ-209-07	Lighting controller	Room 105 - Building 209	HEALTHY	30 Nov 2021	20 May 2027	0	Open -
EQ-209-08	Projector	Room 210 - Building 209	HEALTHY	14 Apr 2022	14 Apr 2027	0	Open -
EQ-216-01	Projector	Room 302 - Building 216	FAULTY	03 Mar 2023	08 Apr 2027	1	Open -
EQ-216-02	Air conditioner	Room 302 - Building 216	HEALTHY	10 Jan 2022	27 Sept 2026	0	Open -
EQ-216-03	Fire detector	Room 302 - Building 216	HEALTHY	15 Jun 2021	08 Dec 2026	0	Open -
EQ-216-04	Door controller	Room 302 - Building 216	HEALTHY	15 Jun 2021	03 Sept 2026	0	Open -
EQ-216-05	Projector	Room 118 - Teaching lab - Building 216	UNDER MAINTENANCE	01 Aug 2024	27 Jan 2027	0	Open -
EQ-216-06	Lighting controller	Level 2 corridor - Building 216	HEALTHY	14 Feb 2020	21 Sept 2026	0	Open -
EQ-216-07	Temperature sensor	Room 118 - Teaching lab - Building 216	HEALTHY	02 Nov 2023	06 Jul 2027	0	Open -
EQ-216-08	Air handling unit	Roof plant room - Building 216	FAULTY	19 Apr 2019	19 Aug 2026	0	Open -
EQ-216-10	Air conditioner	Room 202 - Building 216	UNDER MAINTENANCE	19 Aug 2022	02 Nov 2026	1	Open -
EQ-216-11	Lighting controller	Room 301 - Building 216	HEALTHY	04 Sept 2021	15 Jan 2027	1	Open -

Figure 3 — the equipment register

TAG	TYPE	LOCATION	CONDITION
EQ-209-01	Projector	Workshop 1.04 - Building 209	HEALTHY
EQ-209-02	Air conditioner	Atrium - Building 209	HEALTHY
EQ-209-03	Door controller	Loading bay - Building 209	UNDER MAINTENANCE
EQ-209-04	Fire detector	Basement store - Building 209	FAULTY
EQ-209-05	Lighting controller	Workshop 1.04 - Building 209	DECOMMISSIONED
EQ-209-06	Desktop PC	Room 105 - Building 209	FAULTY
EQ-209-07	Lighting controller	Room 105 - Building 209	HEALTHY
EQ-209-08	Projector	Room 210 - Building 209	HEALTHY
EQ-209-TEST1	Air conditioner	Workshop 1.04 - Building 209	HEALTHY
EQ-216-01	Projector	Room 302 - Building 216	FAULTY
EQ-216-02	Air conditioner	Room 302 - Building 216	HEALTHY
EQ-216-03	Fire detector	Room 302 - Building 216	HEALTHY
EQ-216-04	Door controller	Room 302 - Building 216	HEALTHY
EQ-216-05	Projector	Room 118 - Teaching lab - Building 216	UNDER MAINTENANCE
EQ-216-06	Lighting controller	Level 2 corridor - Building 216	HEALTHY
EQ-216-07	Temperature sensor	Room 118 - Teaching lab - Building 216	HEALTHY
EQ-216-08	Air handling unit	Roof plant room - Building 216	FAULTY

EQ-216-01 FAULTY Edit details Delete

Projector
Room 302 - Building 216
Drop a photo of this unit

INSTALLED: 03 Mar 2023
NEXT SERVICE: 08 Apr 2027
OPEN REQUESTS: 1
SERVICE DUE IN: 194 days
CONSTRUCTION: Faulty

ACTIONS

- Mark faulty**
Makes it on the board and the dashboard
- Under maintenance**
Hold out of use while it is sorted on
- Move unit**
Reassign it to another room
- Return to service**
Clears the fault and the maintenance flag
- Record a service**
Log parts, cost and the next due date
- Decommission**
Retires the asset for good

Every action here writes an entry to the Log Book.

HISTORY

- Lamp module, air filter - 145,000 MMK
7d ago - Daw Htin
- Lamp fault persists after reset - marked faulty
1d ago - Hilda Nee
- Lamp module replaced, 1200h counter reset
15 Feb 2026 - Elysha
- Installed in Room 302
03 Mar 2023 - Facilities team

Figure 4 — a unit's detail drawer, with its history and six actions

```
// src/app/(app)/equipment/page.tsx
// src/lib/equipment-store.ts — every write batches its history row with the change
```

6.4 Sensors

Two jobs on one page: watching the estate, and setting the limits it is watched against.

1. The Monitoring tab shows comfort figures, a floor-by-floor climate grid, and each building's devices.
2. Filter to All devices, Alarms only, or Offline.
3. Select a reading card's chevron to reveal its device id, axes and history.

- The Thresholds tab lists each measuring type's bands, with any room types that disagree shown beneath.
- Edit a band through Manage types. A server room and a lecture hall can hold different limits for the same measurement.

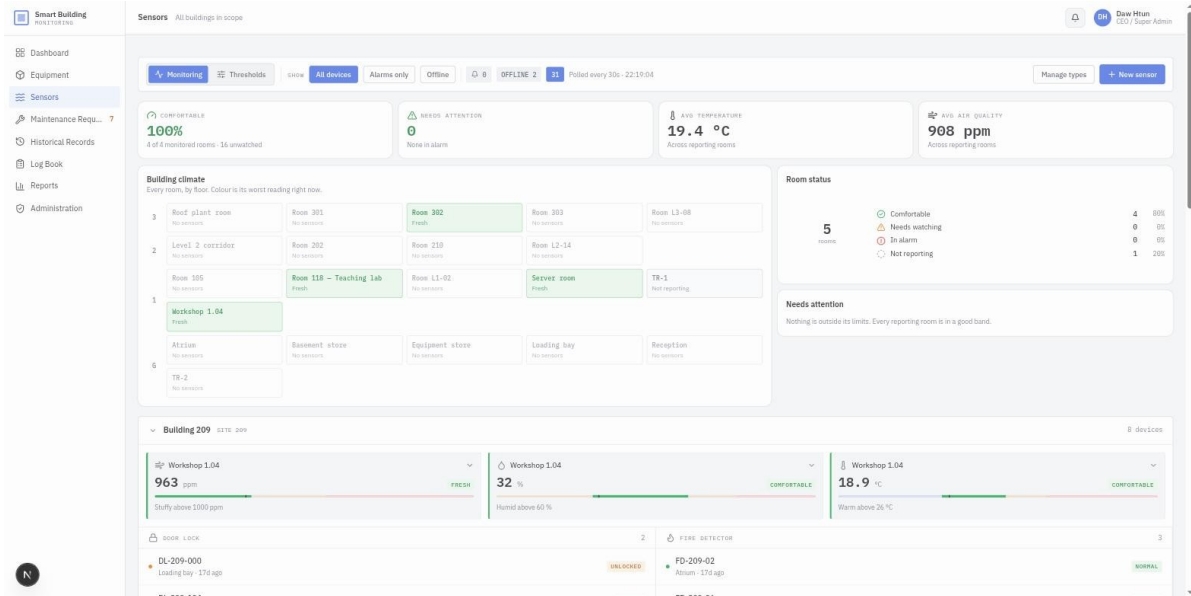


Figure 5 — Monitoring

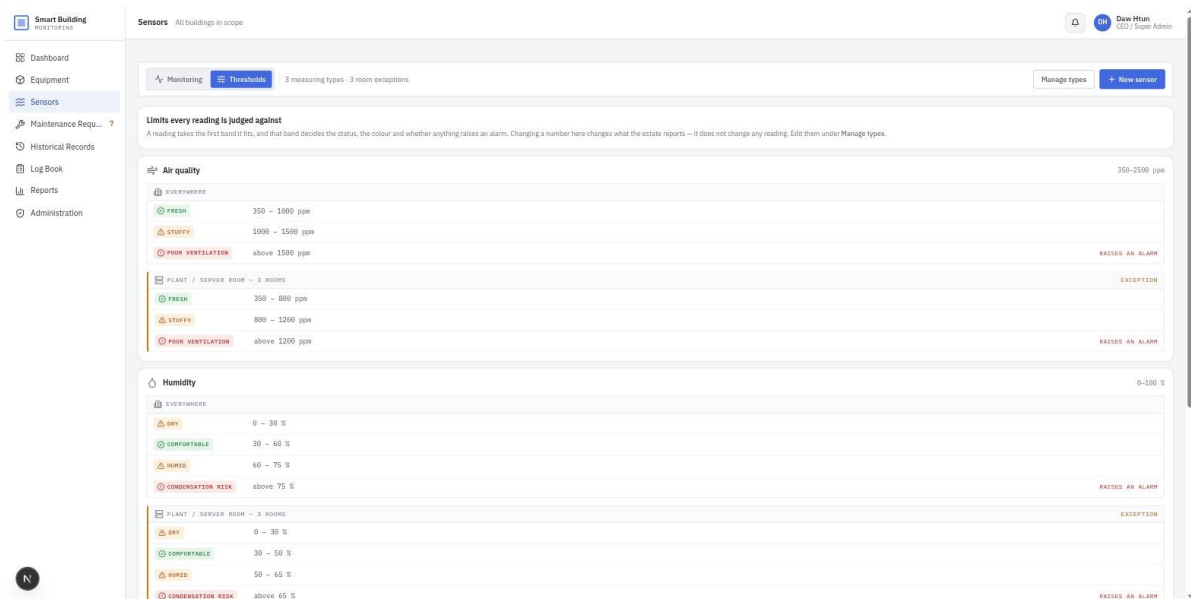


Figure 6 — Thresholds, with room-type exceptions

```
// src/app/(app)/sensors/page.tsx
// src/lib/sensor-readings.ts — where a number becomes a status
// src/lib/climate.ts — the estate rolled up to one row per room
```

6.5 Maintenance requests

The five-step approval workflow, as a board or a table.

1. Select New request. Choose the building, room and equipment unit, describe the fault, and set a priority.
2. The request enters the Requested column.
3. An Admin Manager or the CEO approves it, or sends it back with a written reason.
4. Approved work moves through In progress and Resolved to Completed.
5. The person who raised a request may withdraw it while it is still unapproved, and may flag resolved work as done.

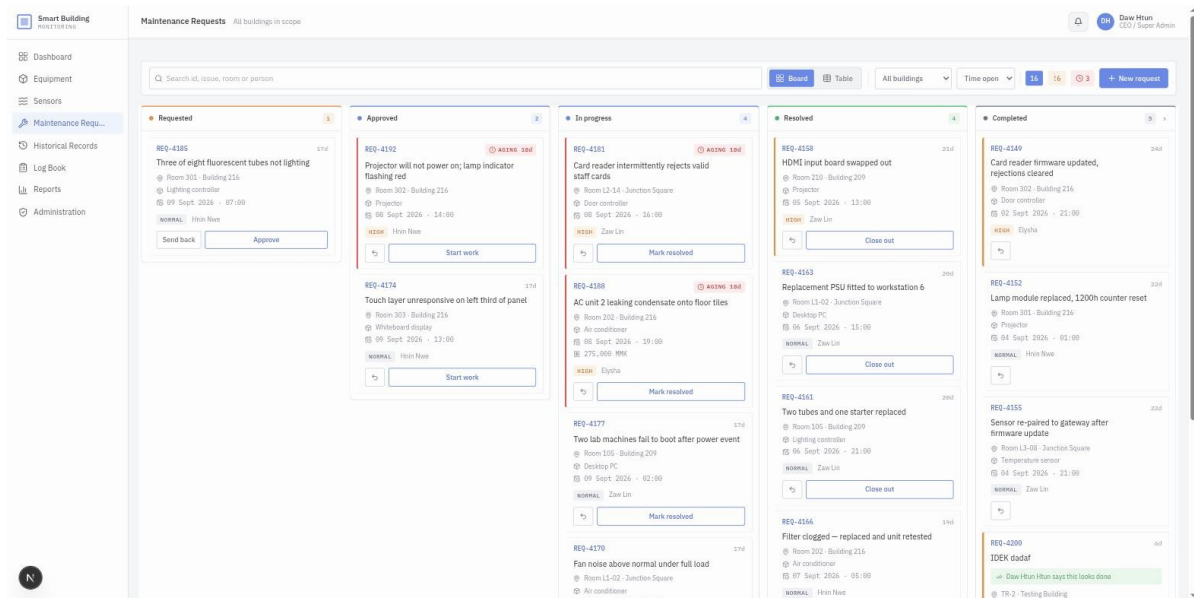


Figure 7 — the five-column board

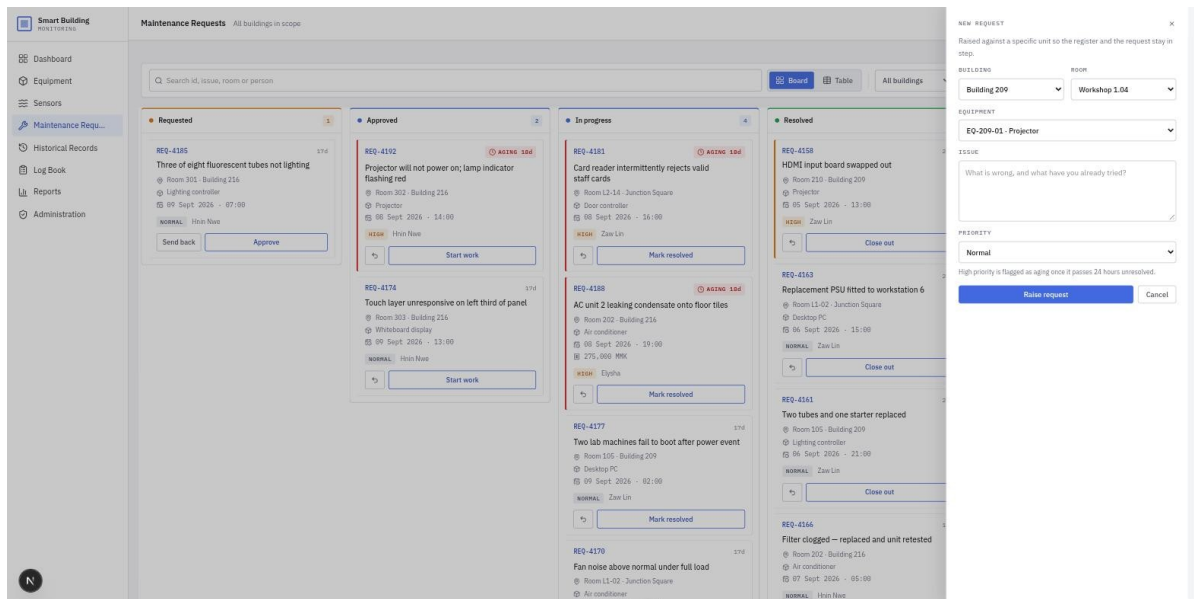


Figure 8 — raising a request

```
// src/app(app)/requests/page.tsx
// moveRequest(id, "next" | "prev") in src/lib/app-state.tsx walks the five steps
```

6.6 Historical records

The significant subset of the audit trail, reachable by every role including Office Staff.

1. Switch between Important and All activity.
2. Narrow by date range, building or record type.
3. Any entry that required a written reason shows that reason beneath the row.
4. Export the filtered set to CSV; the file matches what is on screen.

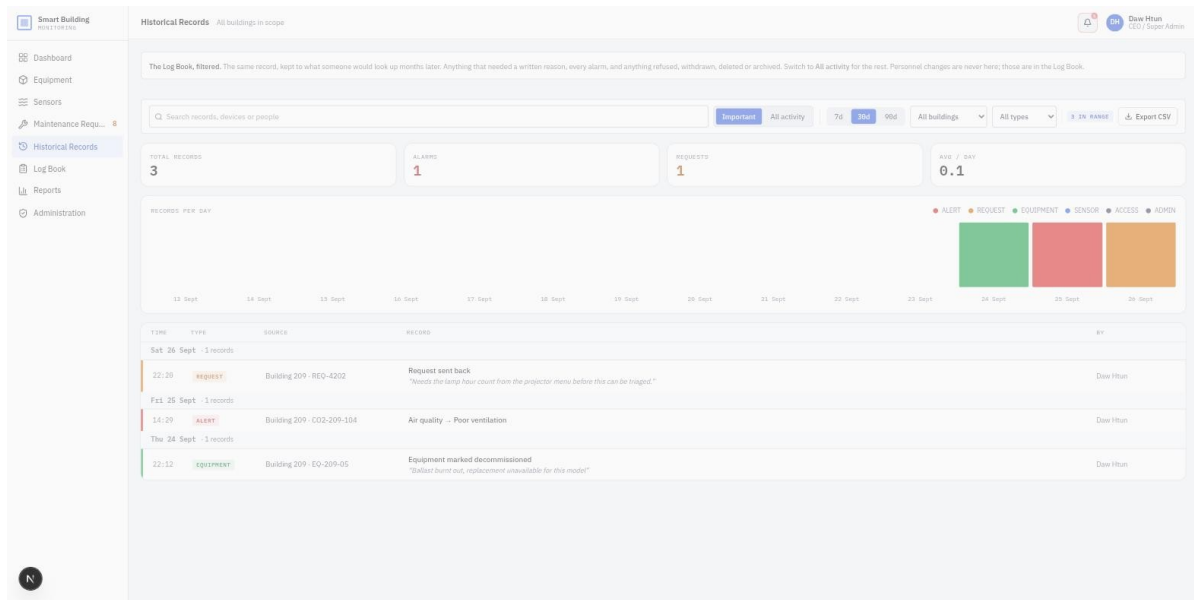


Figure 9 — historical records, reasons shown beneath each row

```
// src/app/(app)/records/page.tsx
// src/lib/records.ts — isSignificant() decides what Important shows
```

6.7 Log Book

The live feed of everything the system writes down. Admin Manager and CEO only.

1. Entries arrive as they happen; use Pause to hold the feed while reading.
2. Actions only is on by default and hides automated sensor readings, which would otherwise be most of the feed.
3. Filter by source or by a date and time range.

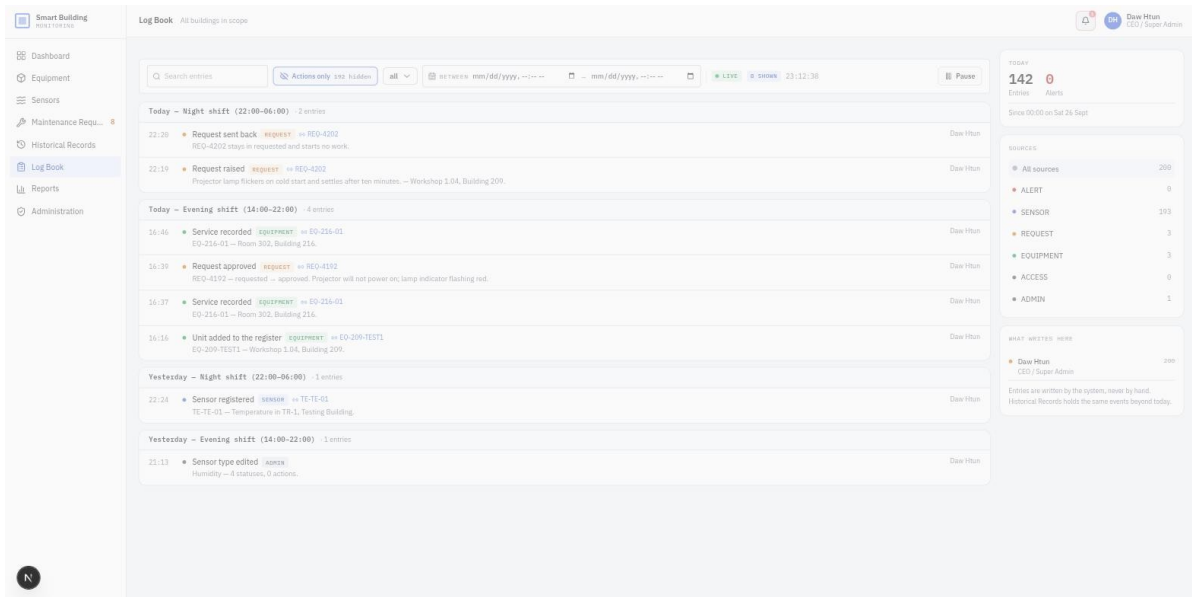


Figure 10 — the Log Book, automated readings hidden

```
// src/app/(app)/logbook/page.tsx
// isRoutineReading() in src/lib/records.ts separates estate chatter from human action
```

6.8 Reports

Generated performance, reliability and cost reports, kept as filed documents.

1. Select Generate report, choose a kind, a scope and a date range.
2. The figures are snapshotted at generation and never recomputed, so a filed report always reads the same.
3. Open a report to read it, or export it straight to PDF or CSV.

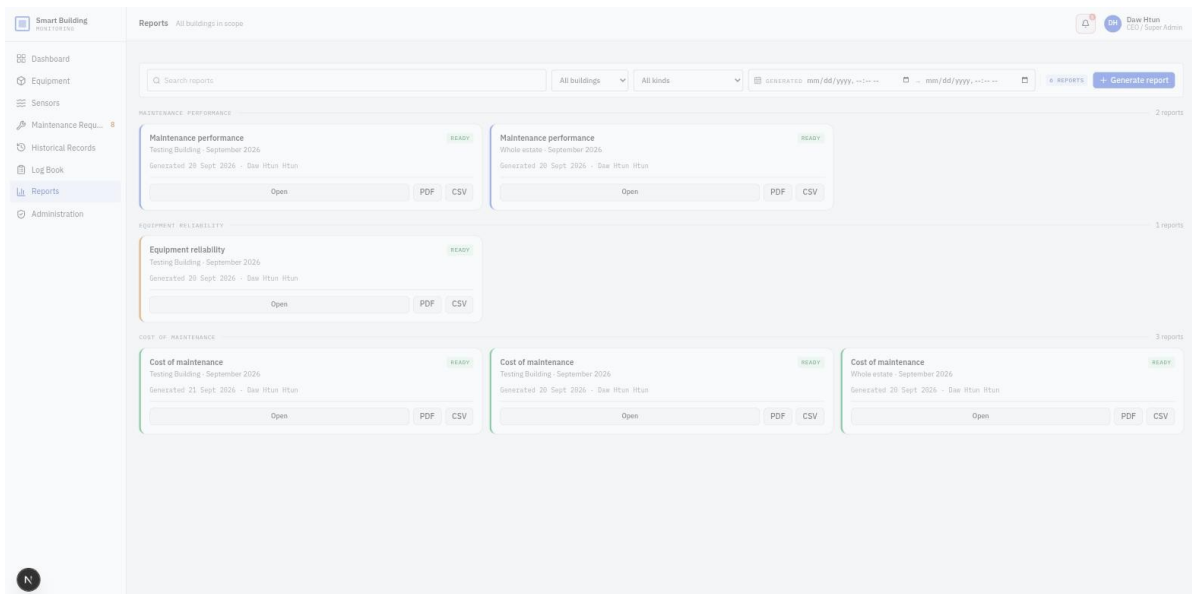


Figure 11 — the report library

```
// src/app/(app)/reports/page.tsx
// src/lib/reporting.ts — buildReport(kind, scope), pure and unit-tested
```

6.9 Administration

Two tabs behind two different permissions. Buildings belongs to the CEO; User Accounts is shared with the Admin Manager.

1. Buildings: add or edit a building, upload a photograph, and manage its rooms. Deleting a building demands a written reason and takes its rooms with it.
2. User Accounts: create an account, change a role, or suspend and restore access.
3. An account is never deleted, only suspended. A suspended person is signed out on their next action.
4. Nobody can edit or suspend their own account; those controls are padlocked on your own row.

NAME	EMAIL	ROLE	ASSIGNED BUILDING	STATUS	LAST ACTIVE	ACTIONS
AMM	aungmyatun.gusto@gmail.com	OFFICE STAFF	Building 209	ACTIVE	6d ago	Edit Suspend
Aung Myat Minn	kaungmyatun0@gmail.com	ADMIN MANAGER	All buildings	ACTIVE	6d ago	Edit Suspend
Aye Chan	aye.chan@university.edu	OFFICE STAFF	Building 216	ACTIVE	8d ago	Edit Suspend
Daw Htun	daw.htun@university.edu	CEO / SUPER ADMIN	All buildings	ACTIVE	Just now	🔒 Edit Suspend
Elysha	elysha@university.edu	ADMIN MANAGER	All buildings	ACTIVE	3d ago	Edit Suspend
Hnin Nwe	hnin.nwe@university.edu	OFFICE STAFF	Building 216	ACTIVE	3h ago	Edit Suspend
Myo Set	myo.set@university.edu	OFFICE STAFF	Junction Square	SUSPENDED	8d ago	Edit Restore
Nay Oo	nay.oo@university.edu	OFFICE STAFF	Junction Square	SUSPENDED	8d ago	Edit Restore
Su Myat	su.myat@university.edu	ADMIN MANAGER	All buildings	SUSPENDED	8d ago	Edit Restore
Thida Win	thida.win@university.edu	OFFICE STAFF	Building 209	SUSPENDED	8d ago	Edit Restore
Zaw Lin	zaw.lin@university.edu	OFFICE STAFF	Building 216	SUSPENDED	8d ago	Edit Restore

Figure 12 — user accounts, with the signed-in user's own row padlocked

```
// src/app/(app)/admin/page.tsx
// canEditUser(actor, target) limits an Admin Manager to Office Staff rows
```

6.10 Settings

1. Your account: change your display name and telephone number.
2. Appearance: choose light, dark or system.
3. Password and sessions: change your password.

```
// src/app/(app)/settings/page.tsx
```

6.11 A note on restricted controls

A control a role may not use is never hidden. It stays on screen behind a padlock, and the padlock explains itself when hovered. A restriction that is visible teaches; one that is hidden simply confuses.

7. Maintenance considerations

7.1 Routine tasks

Task	Frequency	How
Review Firestore usage against the free tier	Monthly	Firebase console, Usage tab. Watch the daily read count in particular
Apply dependency security updates	Monthly	pnpm outdated, then pnpm update. Run the test suite afterwards
Review suspended accounts	Each term	Administration, User Accounts
Export the Log Book for the records	Each term	Historical Records, Export CSV
Verify a restore from backup	Twice yearly	Restore an export into a scratch project and sign in

7.2 Backup and recovery

Cloud Firestore on the free plan has no scheduled export. Two practical options:

- Manual export from the Firebase console before any significant change, which is enough for a project of this size.
- Scheduled daily exports to Cloud Storage, which requires the Blaze pay-as-you-go plan.

The seed script is itself a recovery route for the estate structure. It rewrites buildings, rooms, equipment and sensors to a known state, and leaves anything created through the application untouched:

```
pnpm init:project --dry-run  # show what it would change
pnpm init:project --reset    # clear every document, then rewrite
pnpm init:project --reset-auth # also delete the sign-in accounts
```

7.3 Monitoring

What to watch	Where	Why it matters
Daily reads and writes	Firebase console, Usage	Live subscriptions charge per document. Exceeding the free tier stops the application
Authentication failures	Firebase console, Authentication	A spike suggests either a locked-out user or an attempt to guess passwords
Rules denials	Firebase console, Firestore, Rules monitoring	A denial that surprises you means the two permission layers have drifted apart

Build status	The hosting provider's dashboard	A failed deployment leaves the previous version live, which is safe but stale
--------------	----------------------------------	---

7.4 Known constraints

Constraint	Consequence	Mitigation
No server-side rendering of protected pages	The route guard is a convenience, not security	Authorisation lives entirely in firestore.rules
Photographs stored inside documents	A Firestore document is capped at 1 MiB	Images are downscaled to 640 px and refused above 700 KB
No admin SDK in the project	Accounts cannot be deleted programmatically by another user	Accounts are suspended rather than deleted; --reset-auth deletes only accounts whose password is known
The first user profile cannot be written under the security rules	A fresh project must be seeded before rules are deployed	Documented as Step 6 before Step 7 in the installation procedure
Free-tier daily read limit	A busy day could exhaust it	Upgrade to Blaze, or reduce subscription scope

7.5 Extending the system

Three extension points were designed in and need no schema migration:

- Equipment and sensor types are data, not code. A new kind of asset or device is added through the type registry in the interface.
- Sensor thresholds, including per-room-type exceptions, are edited in the interface rather than in a deployment.
- Adding a building or a room in Administration reaches every building filter in the application at once.

Changes that do require a developer: adding a fourth role, adding a field to an existing document type, or adding a query that filters and sorts on different fields, which would need a composite index.

7.6 Handover checklist

1. Transfer ownership of the Firebase project to the incoming maintainer's Google account.
2. Transfer or recreate the hosting account, and re-enter the six environment variables there.
3. Hand over the repository, including the .env.example file but never .env.local.
4. Confirm the incoming maintainer can sign in, run pnpm dev, and deploy the rules.
5. Walk through one full request lifecycle together, from raising to completion.